Evolutionary Object Graph: A Seed-Driven, Evidence-Bounded Graph Operating Model for AI-Assisted Specification Work

Author: Rim Zabarov

Affiliation: Independent Researcher

Email: rim@zabarov.com

ORCID: https://orcid.org/0009-0004-8605-5594

Working Paper, Version 1.3

Date: 2026-05-28

Abstract

Large language model agents often rely on text-first knowledge substrates: prompts, documentation files, technical specifications and chat histories. In evolving software projects, this approach becomes brittle because requirements, features, initiatives, implementation constraints, evidence and governance decisions are distributed across many artifacts. This paper proposes the Evolutionary Object Graph (EOG), a seed-driven and evidence-bounded graph operating model for AI-assisted specification and skill-knowledge workflows. EOG integrates typed graph memory, requirements traceability, provenance, lifecycle/readiness states, governance gates and generated context packs into a single design artifact. Unlike GraphRAG methods focused primarily on retrieval, EOG separates canonical graph state from generated context and requires evidence/review gates before canonical updates or runtime actions. Internal case studies show substantial context-size reductions across several skill and project-knowledge workflows, including an interdependent specification benchmark. The current evidence supports context-reduction and design-feasibility claims only. Semantic completeness, external validity and public reproducibility remain explicit validation targets.

Keywords: knowledge graph; GraphRAG; agent memory; requirements traceability; AI governance; design science; provenance; AI-assisted specification; context generation

AI Assistance Disclosure: AI tools were used to assist with drafting, structuring, literature-positioning notes, figure/table preparation and project-organization tasks. The author reviewed and remains responsible for the manuscript, claims, limitations, source selection and final submission decisions.

1. Introduction

AI-assisted software and knowledge work increasingly depends on agents that read, interpret and act on large bodies of project-specific text. These agents often use long prompts, documentation folders, technical specifications, chat histories and rule files as their working knowledge substrate. This approach works while the knowledge base is small. It becomes fragile when the system evolves across many artifacts, packages, features and decisions.

The practical origin of this work was an internal multi-package technical specification workspace with more than twenty interdependent software packages. Each package required a technical specification, but the specifications were not independent. Features in one package affected features in other packages. New product initiatives changed the meaning, priority and implementation logic of existing features. A text-first AI workflow could read files, but it could not reliably preserve package-to-package dependencies, feature-to-feature relations, evidence, rationale, readiness and action constraints across the whole evolving system.

An intermediate attempt represented specification knowledge through machine- readable arrays. This improved explicit structure but did not solve the deeper problem. The number of arrays increased, and the AI still had to infer which array represented which layer, which relation was current, which object was source truth, and which generated artifact was only a view. The problem was not only data format. It was cross-layer relation management under continuous change.

This paper proposes the Evolutionary Object Graph (EOG) as a response to that problem. EOG is a graph operating model in which requirements, features, initiatives, decisions, evidence, risks, actions, readiness states and governance constraints are represented as graph-addressable objects connected by typed relations. The model is evolutionary because a graph is not treated as a static database. Each graph can have a purpose, Graph DNA, growth rules, profiles, feedback loops, readiness gates and seed-driven expansion.

The paper does not claim that knowledge graphs, graph retrieval, graph memory, requirements traceability or AI governance are new. The contribution is an integration artifact: a design-science model that combines these layers for AI-assisted specification and skill-knowledge work, with explicit boundaries between canonical state, generated context, evidence and permission to act.

2. Related Work And Positioning

GraphRAG has already established graph-structured retrieval as a major direction in retrieval-augmented generation (Peng et al., 2024). Agentic GraphRAG further extends this field toward graph-native agents and graph operations as part of agent workflows (Chen et al., 2026). Graph-based agent memory has also become an active research direction for long-horizon LLM agents, emphasizing relational memory, retrieval and memory evolution (Yang et al., 2026). AriGraph demonstrates that LLM agents can construct and update knowledge graph world models that combine semantic and episodic memory (Anokhin et al., 2024).

These directions are close to EOG but do not exhaust the problem addressed here. GraphRAG and graph memory primarily address retrieval, memory and agent reasoning. EOG uses related ideas, but its intended scope is an operational layer for evolving

specifications, skills and project knowledge: objects have lifecycle/readiness states, evidence boundaries, governance constraints and generated context artifacts.

Requirements traceability is another important foundation. Traceability research already studies relations among requirements, design artifacts, code and tests, including formal semantics for consistency checking and inference (Goknil et al., 2011). Knowledge graphs are also already used in software engineering for multiple applications, including software knowledge organization and intelligent development support (Wang et al., 2023). EOG therefore does not claim novelty for software-engineering knowledge graphs or requirements relations. Its gap is the combination of traceability with LLM- agent context generation, evidence/readiness gates and graph-governed action constraints.

Provenance is also an established area. PROV-O provides a W3C ontology for representing provenance information across systems (Lebo et al., 2013). EOG should be interpreted as compatible with this concern rather than as an attempt to replace provenance standards. Its specific contribution is to make source/evidence boundaries part of a working graph used by AI agents to generate context and decide whether action is allowed.

Multi-agent LLM frameworks such as AutoGen demonstrate customizable agents that combine LLMs, tools and human participation (Wu et al., 2023). EOG is not proposed as a general replacement for such frameworks. It is proposed as a graph-structured knowledge, evidence and governance substrate that could support or constrain agents operating in such frameworks.

Finally, AI governance and risk management are already covered by frameworks and standards such as the NIST AI Risk Management Framework and ISO/IEC 42001 (Tabassi, 2023; International Organization for Standardization, 2023). EOG does not claim novelty for AI governance itself. Its governance claim is operational: policies, readiness, approval, fallback and action limits can be represented as graph objects that affect context generation and action permission.

EOG differs from these adjacent approaches in five concrete ways.

1. GraphRAG primarily improves retrieval and grounding for generation, while EOG coordinates evolving specification work through graph state, generated context and review gates.
2. Requirements traceability preserves links among requirements and engineering artifacts, while EOG also treats evidence, readiness, generated context and permission to act as graph-addressable objects.
3. Governance frameworks define risk, accountability and management controls, while EOG operationalizes selected controls inside the working graph used by AI agents.
4. EOG separates canonical graph state from generated context packs. A context pack can support a task without becoming source truth.
5. EOG separates context selection from action permission. A graph may reduce the context required for planning while still blocking canonical writes or runtime actions until evidence and review gates are satisfied.

3. Research Problem

The central problem is the degradation of text-first AI context management in complex evolving knowledge systems. When project knowledge is distributed across documents, chats, specifications, rules and implementation notes, an AI agent must infer not only

facts but also relations, source boundaries, status and action constraints. As the system grows, this inference becomes unreliable.

The practical failure mode can be stated as follows:

> Text-first and array-first AI workflows struggle to preserve cross-layer relations among requirements, features, initiatives, evidence, readiness, decisions and action constraints as systems evolve.

The research question for this first paper is:

> Can a seed-driven Evolutionary Object Graph provide a more reliable operating substrate for AI-assisted specification work by preserving typed relations, lifecycle, evidence, readiness, governance and profile-specific context generation?

The current hypothesis is conservative:

> In internal AI-assisted specification and skill-maintenance workflows, graph-structured context can reduce context-selection volume and make evidence and review boundaries more explicit. Broader semantic quality, external validity and large-scale governance claims require further validation.

4. Methodology

This paper follows a design-science framing. Design science research builds and evaluates artifacts that extend human and organizational capabilities (Hevner et al., 2004). EOG is treated as such an artifact: a proposed graph operating model derived from an observed practical problem and evaluated through internal case-study evidence.

The study combines four methods:

1. Problem-origin analysis from an internal interdependent specification case.
2. Related-work comparison against GraphRAG, agent memory, requirements traceability, provenance, multi-agent orchestration and AI governance.
3. Conceptual artifact design: the EOG model, operating loops, seed layer and governance boundaries.
4. Internal benchmark analysis of graph-mode context packaging across skill and project-knowledge cases.

The current evidence is internal and should be interpreted as early design-science evidence rather than broad empirical validation. The primary quantitative outcome is context-size reduction. Semantic quality, task success and external validity are treated as future validation targets, not proven outcomes.

For the context-size measurements, `baseline context` denotes the broader document or context set that would be supplied in the text-first/current workflow for a given task. `Graph context` denotes the context package selected or generated through the graph-mode profile for the same or equivalent task. The reduction metric is calculated as:

```
(baseline_context_size − graph_context_size) / baseline_context_size ∗
100
```

The compared tasks were internal specification, skill-maintenance or context-selection tasks from the same workflow families. The measurement is therefore a context-selection efficiency metric, not a direct measure of answer correctness, semantic completeness, implementation quality or safety. Those outcomes require independent evaluation.

5. The Evolutionary Object Graph Model

EOG represents meaningful project and agent-workflow elements as typed graph objects. These can include requirements, features, initiatives, packages, rules, scenarios, decisions, sources, artifacts, evidence, events, signals, actions, feedback, learning, policies and governance objects. Relations are first-class: the model is designed to preserve what depends on what, what evidence supports a claim, what action is blocked, and which generated context comes from which canonical source.

The model can be described as a layered stack:

```
Object Layer        what exists
Knowledge Layer     what is known
Relation Layer      how things are connected
Lifecycle Layer     where each object is in development
Initiative Layer    why change happens
Evidence Layer      how claims are supported
Action Layer        what can be done
Governance Layer    who has authority and responsibility
Federation Layer    how graphs interact with larger graphs
```

The primary operating loop is:

```
intake -> classify -> extract -> link -> validate -> impact -> plan ->
evidence -> accept -> evolve
```

This loop is intentionally review-bounded. Generated artifacts such as indexes, context packs and reports are not source truth. They are derived views. Canonical graph state must be updated through evidence and review.

This separation matters for AI agents. A graph may generate a smaller context pack for a task, but that does not mean the agent is allowed to edit source truth or perform a runtime action. Context selection and permission to act are different decisions.

A compact formalization of the current EOG model is:

```
EOG = (O, R, E, S, P, G, C)
```

where `O` is a set of typed graph objects, `R` is a set of typed relations, `E` is a set of evidence references, `S` is a set of lifecycle/readiness states, `P` is a set of graph profiles, `G` is a set of governance gates and `C` is a set of generated context packs. A graph object minimally contains an identifier, kind, name, readiness state and evidence references. A relation minimally contains source object, relation type, target object and supporting evidence. A context pack is a generated projection over selected objects and relations; it is not canonical graph state. A governance gate can permit, block or require review before canonical writes or runtime actions.

Typical object kinds include `requirement`, `feature`, `package`, `decision`, `risk`, `evidence`, `policy`, `action`, `context_pack` and `review_gate`. Typical relation types include `depends_on`, `supports`, `blocks`, `requires`, `impacts`, `derived_from`, `triggers` and `has_risk`. Typical readiness states include `draft`, `review_needed`, `approved`, `blocked`, `critical` and `deprecated`. The exact taxonomy is profile-dependent, but the invariant is that claims, relations and generated views remain evidence-bounded.

The current model assumes five operational invariants:

1. A canonical object cannot be updated without evidence or an explicit review state.
2. A generated context pack cannot become source truth automatically.
3. Runtime action permission must be evaluated separately from context relevance.
4. Relations that affect implementation or governance require supporting evidence.
5. Profiles may change projection scope, but they must not silently change canonical graph truth.

The layered model can be visualized as an integration across memory, traceability, evidence, readiness, action and governance. The model does not imply that all layers are empirically validated in the current study; it describes the proposed artifact architecture.

6. Seed-Driven Growth

Large graphs can become too complex for direct human inspection. EOG therefore introduces a seed-driven control layer. A human does not have to design or review the whole graph at once. Instead, the human defines a seed:

- purpose;
- target profile;
- source boundaries;
- allowed object families;
- allowed relation families;
- growth budget;
- review gates;
- stop conditions.

The seed produces a first candidate graph, or Graph Embryo. The embryo is small, reviewable and disposable. It is not canonical truth. If the embryo is accepted, it can inform canonical graph growth. If it fails, the seed can be changed before deeper extraction or writing occurs.

The seed-driven model is intended to reduce the practical cognitive burden of working with large graph systems. Instead of requiring full-graph comprehension, it shifts human control to seed design, profile selection, projection views, readiness checks and review gates.

The seed-growth process can be visualized as a loop from Graph Seed to Graph Embryo, review, canonical graph, generated views and learning. The important boundary is that an embryo is a candidate preview, not source truth.

7. Illustrative Synthetic Example

A public-safe synthetic example was prepared to make the mechanism inspectable without exposing private source material. In the example, a fictional modular publishing platform contains four packages: identity/access, content workflow, audit events and notification delivery. A task asks the agent to prepare implementation context for sending notifications after content approval.

In a text-first workflow, the agent would likely receive all four source notes. In EOG, the task can be represented through graph objects and relations:

```
pkg.content_workflow
  defines -> feature.workflow_transition

pkg.audit_events
  records -> feature.workflow_transition

feature.workflow_transition
  triggers -> pkg.notification_delivery

pkg.notification_delivery
  requires -> feature.scoped_api_token
  blocked_by -> constraint.audit_before_notify
  has_risk -> risk.duplicate_notifications

constraint.audit_before_notify
  depends_on -> pkg.audit_events
```

The resulting compact context pack contains the workflow transition, audit package, audit-before-notify constraint, notification delivery package, scoped-token dependency and duplicate-notification risk. It also includes review gates: audit persistence before notification enqueue, scoped token usage and idempotency/correlation handling. This example illustrates the central EOG distinction: the generated context pack gives the AI task-relevant working context, while the canonical graph and governance gates still control what can be changed or executed.

The same example can be represented as a compact context pack:

```
Context pack CP-001: Notifications After Approval

Selected objects:
- feature.workflow_transition
- pkg.audit_events
- constraint.audit_before_notify
- pkg.notification_delivery
- feature.scoped_api_token
- risk.duplicate_notifications

Required gates:
- audit persistence before notification enqueue
- scoped token usage
- idempotency or correlation handling
```

```
Boundary:
- generated context for implementation planning
- not canonical source truth
- no runtime action permission by itself
```

The synthetic package also includes an illustrative metric calculation:

Synthetic example measure	Value
Text-first context	4 source notes
Baseline context units	620
Graph context units	270
Illustrative reduction	56.45%

In the synthetic package, context units are illustrative normalized units used only to demonstrate the reduction calculation.

These synthetic values demonstrate calculation format only. They are not merged with the internal empirical results below.

8. Internal Case-Study Evidence

The strongest current empirical evidence concerns context-size reduction. Across internal skill and project-knowledge workflows, graph-mode context packs substantially reduced selected context size compared with broader current-mode context selection.

Table 1. Internal context-size reduction measurements across graph-mode workflows.

Case	Baseline context	Graph context	Reduction	Records/tasks
EOG self-benchmark	305,417	76,439	74.97%	10
Marketing skill pilot	171,500	68,252	60.20%	6
Content skill pilot	50,436	19,581	61.18%	6
Ops skill pilot	215,018	19,227	91.06%	6
Tester skill pilot	246,056	17,497	92.89%	6
UX skill pilot	134,489	30,094	77.62%	7
Docs skill pilot	212,351	41,522	80.45%	9
Interdependent spec benchmark	726,994	55,429	92.38%	6

The interdependent spec benchmark is the most important case because it matches the original problem: evolving multi-package technical specifications. In that case, the compact graph profile reduced selected context from `726994` to `55429`. Domain review marked the compact profile as useful for first-pass routing and impact orientation, while requiring a standard profile before canonical technical specification editing.

These results support a narrow claim: graph-mode context packaging can reduce selected context size in internal AI-assisted specification and skill- knowledge workflows. They do not prove semantic completeness or general task success. Several semantic metrics in the evidence package are heuristic and require independent human validation.

Table 1 is interpreted as internal case-study evidence. It reports context-size measurements, not independent task-success outcomes. It is therefore suitable for a design-science working paper, but not sufficient for claims of broad empirical superiority.

9. Governance, Readiness And Safety Boundaries

A central design choice in EOG is that measurement does not equal permission. Context reduction may make AI work more efficient, but it does not authorize unsafe action. The graph model therefore includes readiness states, policy guards, fallback rules and approval gates as graph-addressable elements.

This design is consistent with broader AI governance concerns: AI systems need risk management, accountability, transparency and operational controls (Tabassi, 2023; International Organization for Standardization, 2023). EOG does not replace these governance frameworks. It attempts to operationalize some of their concerns inside the working graph used by AI agents.

For example, a context pack may be allowed for planning, but canonical writes can remain blocked. A compact profile may be approved for impact orientation, but a broader profile may be required for final design decisions. Generated reports can support review, but they do not become source truth automatically.

10. Limitations

This draft has several limitations.

First, the evaluation is internal. It uses private skill and specification workflows. External validity is not yet established.

Second, the main quantitative result is context-size reduction. This is useful for AI-assisted work, but it is not equivalent to correctness, completeness, task success or safety.

Third, semantic scores in the current evidence package are heuristic in several places. They should be treated as review signals, not validated outcome measures.

Fourth, public reproducibility requires a synthetic or fully redacted dataset because some source materials come from private internal project repositories. A draft synthetic package now exists for method inspection, but it still requires review before it can support a public reproducibility claim.

Fifth, large-scale claims about organizations, institutions or state-level systems are research horizons only. They are not demonstrated by the current evidence.

Sixth, the current project includes a draft synthetic benchmark package for method inspection. It is not yet a validated public replication package, and it must not be merged with the internal empirical benchmark results.

Data Availability

The current study uses internal design artifacts, benchmark records and context-pack measurements from private skill and specification repositories. Aggregate context-size metrics and methodological summaries are reported in this manuscript. The underlying raw source materials include internal project and skill documentation and are not publicly released.

A limited public evidence package is planned. Its recommended first layer contains aggregate metrics, metric definitions, figure and table sources, and a synthetic benchmark corpus that reproduces the structure of the method without exposing private internal project source materials. A draft synthetic package has been prepared for method inspection, but the empirical evidence should still be described as internal case-study evidence until the package is reviewed and released.

The synthetic package is structured as follows:

```
source-corpus/      synthetic source notes
graph/objects.json  typed graph objects
graph/relations.json typed graph relations
tasks/tasks.json    context-selection task definition
expected/           expected compact context pack
metrics/            illustrative metric calculation
scripts/            repeatable context-reduction calculation
```

Run: `node scripts/calculate-context-reduction.js`.

The synthetic package supports method inspection only. It does not reproduce the internal empirical benchmark and must not be interpreted as external validation.

Declarations

Author Contributions: Rim Zabarov is the sole author of the current manuscript draft. The author developed the Evolutionary Object Graph concept, collected and organized the internal case-study materials, prepared the manuscript and is responsible for the reported interpretation and limitations.

Funding: No external funding was received for this work.

Conflict Of Interest: The author developed and maintains the internal skill and specification workflows used as case-study evidence. This relationship should be disclosed because the empirical materials come from the author's own applied research and development environment.

Ethics Statement: The current manuscript does not report experiments on human participants, animals, medical data, biometric data or protected personal data. It uses internal project and specification artifacts as case-study materials. Future studies involving organizational users, agent performance monitoring or human-task evaluation should receive a separate ethics and privacy review.

11. Future Work

Future work should proceed in five steps.

1. Review and release a synthetic public benchmark package for method inspection.
2. Validate semantic quality through independent human review.
3. Measure task outcomes, not only context size.
4. Run the seed cognitive-load protocol comparing raw-source review, full- graph review and seed-preview review.
5. Extend related work with ontology engineering, SHACL/RDF/OWL, management cybernetics and RAG evaluation literature if those claims remain in the final manuscript.

12. Conclusion

The Evolutionary Object Graph is proposed as a seed-driven, evidence-bounded graph operating model for AI-assisted specification and skill-knowledge work. Its contribution is integrative: it combines graph memory, requirements traceability, provenance, lifecycle/readiness, governance gates, seed-driven growth and generated context packs into one design artifact.

Initial internal case studies show substantial context-size reductions, with the interdependent specification benchmark providing an origin-aligned example of the problem and potential solution. A draft synthetic package now provides a safer path toward public method inspection without exposing private source material. The current evidence is promising but bounded. EOG should be developed as a design-science research program, with public reproducibility, semantic validation and external evaluation as the next critical gates.

References

Anokhin, P., Semenov, N., Sorokin, A., Evseev, D., Kravchenko, A., Burtsev, M., & Burnaev, E. (2024). *AriGraph: Learning knowledge graph world models with episodic memory for LLM agents*. arXiv. https://arxiv.org/abs/2407.04363

Chen, Z., Zheng, L., & Zhu, D. (2026). *A survey of Agentic GraphRAG: From retrieval-augmented generation to graph-native agents*. SSRN. https://ssrn.com/abstract=6713979 https://doi.org/10.2139/ssrn.6713979

Goknil, A., Kurtev, I., van den Berg, K., & Spijkerman, W. (2011). Semantics of trace relations in requirements models for consistency checking and inferencing. *Software and Systems Modeling*. https://doi.org/10.1007/s10270-009-0142-3

Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. *MIS Quarterly, 28*(1), 75-106. https://doi.org/10.2307/25148625

International Organization for Standardization. (2023). *ISO/IEC 42001:2023: Information technology - Artificial intelligence - Management system*. https://www.iso.org/standard/42001

Lebo, T., Sahoo, S., & McGuinness, D. (Eds.). (2013). *PROV-O: The PROV Ontology*. W3C Recommendation. https://www.w3.org/TR/prov-o/

Peng, B., Zhu, Y., Liu, Y., Bo, X., Shi, H., Hong, C., Zhang, Y., & Tang, S. (2024). *Graph retrieval-augmented generation: A survey*. arXiv. https://arxiv.org/abs/2408.08921

Tabassi, E. (2023). *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*. National Institute of Standards and Technology. https://doi.org/10.6028/NIST.AI.100-1

Wang, L., Sun, C., Zhang, C., Nie, W., & Huang, K. (2023). Application of knowledge graph in software engineering field: A systematic literature review. *Information and Software Technology, 164*, Article 107327. https://doi.org/10.1016/j.infsof.2023.107327

Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., Jiang, L., Zhang, X., Zhang, S., Liu, J., Awadallah, A. H., White, R. W., Burger, D., & Wang, C. (2023). *AutoGen: Enabling next-gen LLM applications via multi-agent conversation*. arXiv. https://arxiv.org/abs/2308.08155

Yang, C., Zhou, C., Xiao, Y., Dong, S., Zhuang, L., Zhang, Y., Wang, Z., Hong, Z., Yuan, Z., Xiang, Z., Chen, S., Zhou, H., Zhang, Q., Liu, N., Su, J., Wang, X., Chang, Y., & Huang, X. (2026). *Graph-based agent memory: Taxonomy, techniques, and applications*. arXiv. https://arxiv.org/abs/2602.05665